Tests Unitaires avec Jest

Jest est un framework de tests pour Javascript.
https://jestjs.io/docs/getting-started

Tests Unitaires avec Jest.....1
1. Installer/Configurer Jest..... 1
1.1. Créer le fichier de configuration de Jest.....1
1.2. Modifier tsconfig.json.....3
1.3. Créer le fichier tsconfig.spec.json..... 3
2. Ajouter un test simple.....3
2.1. Créer un dossier __tests__..... 3
2.2. Créer un fichier sample.test.ts.....3
2.3. Lancer les tests..... 4
3. Rappel sur les tests unitaires..... 4
4. Arrange - Setup and Teardown..... 5
5. Assert - Matchers Jest.....5
5.1. Valeurs simples..... 5
5.2. Strings..... 6
5.3. Nombres.....6
5.4. Tableaux..... 6
5.5. Exceptions.....6
5.6. Mocks – Vérifier les appels et isoler les dépendances.....7
5.6.1. Créer un mock avec jest.fn().....7
5.6.2. Mocker un module entier avec jest.mock().....9
6. Mocker une dépendance externe (base de données).....10
6.1. Analyse de la méthode createVisiteur..... 11
6.2. Code des tests..... 13

1. Installer/Configurer Jest

npm install --save-dev jest ts-jest @types/jest

1.1. Créer le fichier de configuration de Jest

Il peut aussi être créé via l'assistant, mais il y a des modifications à faire sur le fichier.

→ **jest.config.ts (à la racine du projet)**

```
import type {Config} from 'jest';
const config: Config = {
```

```
  // Utilise ts-jest pour compiler les fichiers TypeScript
  preset: 'ts-jest',
  // Environnement Node. js (pas navigateur)
  testEnvironment: 'node',

  // Dossiers où chercher les tests
  roots:  ['<rootDir>/src'],
  // Patterns de fichiers de test
  testMatch: [
    '**/__tests__/**/*.ts',
    '**/?(*.)+(spec|test).ts'
  ],
  globals: {
    'ts-jest': {
      tsconfig: 'tsconfig.spec.json',
    },
  },

   // Nettoyer automatiquement les mocks avant chaque test
  clearMocks: true,
  // Collecter la couverture de code
  collectCoverage: true,
  // Fichiers à inclure dans la couverture
  collectCoverageFrom: [
    'src/**/*.ts',
    '!src/**/*.d.ts',
    '!src/**/*.interface.ts',
    '!src/**/__tests__/**',
  ],

  // Dossier de sortie pour la couverture
  coverageDirectory: 'coverage',
  // Provider de couverture
  coverageProvider: 'v8',
  // Extensions de fichiers reconnues
  moduleFileExtensions:  ['ts', 'js', 'json'],
  // Afficher les détails des tests
  verbose: true,
};
export default config;
```

1.2. Modifier tsconfig.json

Modifier le fichier tsconfig.json afin d'exclure les dossiers de tests.

```
"exclude": ["node_modules", "dist", "src/__tests__"]
```

1.3. Créer le fichier tsconfig.spec.json

Créer un fichier tsconfig.spec.json pour ajouter Jest au projet.
Cela permet d'éviter de pouvoir utiliser des méthodes du framework Jest dans le code source.

```
{
 "extends": "./tsconfig.json",
 "compilerOptions": {
   "types": ["node", "jest"],
   "noEmit": true
 },
 "include": ["src/**/*.ts"]
}
```

2. Ajouter un test simple

2.1. Créer un dossier __tests__

Créer le dossier __tests__ dans src. C'est ici que seront créées les classes de test.
Structure du projet :

2.2. Créer un fichier sample.test.ts

Créer un fichier sample.test.ts dans le dossier __tests__ avec le code suivant :

```
describe('Configuration Jest', () => {
  test('devrait additionner correctement', () => {
    expect(1 + 1).toBe(2);
  });

  test('devrait gérer les chaînes', () => {
    const message = 'Hello Jest';
    expect(message).toContain('Jest');
  });
});
```

La méthode *describe* permet de grouper des tests.
La méthode *test* permet de définir un test à exécuter.

2.3. Lancer les tests

Pour lancer les tests : **npm run test**

3. Rappel sur les tests unitaires

Le **pattern AAA** est la structure recommandée pour écrire des tests clairs et maintenables.

Étape	Signification	Action
ARRANGE	Préparer	Configurer/créer les données et mocks de test
ACT	Agir	Exécuter la fonction à tester
ASSERT	Vérifier	Contrôler que le résultat est correct

Exemple :

- **Fonction à tester**

```
function calculerTotal(prix: number, quantite: number): number {
  return prix * quantite;
}
```

- **Test**

```
test('calcule le total correctement', () => {
  // ========== ARRANGE ==========
  // Préparer les données d'entrée
  const prix = 10;
```

```
  const quantite = 3;
  const totalAttendu = 30;

  // ========== ACT ==========
  // Exécuter la fonction
  const result = calculerTotal(prix, quantite);

  // ========== ASSERT ==========
  // Vérifier le résultat
  expect(result).toBe(totalAttendu);
});
```

4. Arrange - Setup and Teardown

https://jestjs.io/docs/api#reference

Si vous devez initialiser des valeurs, instancier des objets avant les tests. Les méthodes appelées avant les tests sont nommées *Setup* et après les tests *Teardown*.

Méthode	Quand	Usage
`beforeEach(fn)`	Avant **chaque** test	Réinitialiser les mocks
`afterEach(fn)`	Après **chaque** test	Nettoyer
`beforeAll(fn)`	Avant **tous** les tests	Setup lourd (connexion DB)
`afterAll(fn)`	Après **tous** les tests	Fermer connexions

5. Assert - Matchers Jest

https://jestjs.io/docs/expect#matchers

Les matchers permettent de vérifier le résultat attendu afin que le test soit réussi.
Assertions (expect)

5.1. Valeurs simples

Assertion	Usage	Exemple
`.toBe(value)`	Égalité stricte (===)	`expect(x).toBe(5)`
`.toEqual(value)`	Égalité profonde (objets)	`expect(user).toEqual({nom: 'Dupont', age: 30 })`

`.toBeDefined()`	Valeur définie	`expect(result).toBeDefined()`
`.toBeNull()`	Valeur null	`expect(x).toBeNull()`
`.toBeTruthy()`	Valeur truthy	`expect(x).toBeTruthy()`
`.toBeFalsy()`	Valeur falsy	`expect(x).toBeFalsy()`

5.2. Strings

Assertion	Usage	Exemple
`.toContain(substring)`	Contient une chaîne	`expect(text).toContain('hello')`
`.toMatch(regex)`	Correspond à regex	`expect(email).toMatch(/.*@.*/)`

5.3. Nombres

Assertion	Usage	Exemple
`.toBeGreaterThan(n)`	Plus grand que	`expect(x).toBeGreaterThan(10)`
`.toBeLessThan(n)`	Plus petit que	`expect(x).toBeLessThan(5)`
`.toBeCloseTo(n, digits)`	Proche de (floats)	`expect(0.1 + 0.2).toBeCloseTo(0.3)`

5.4. Tableaux

Assertion	Usage	Exemple
`.toHaveLength(n)`	Longueur du tableau	`expect(arr).toHaveLength(3)`
`.toContain(item)`	Contient un élément	`expect(arr).toContain('apple')`

5.5. Exceptions

Assertion	Usage	Exemple
`.toThrow()`	Lance une erreur	`expect(() => fn()).toThrow()`
`.toThrow(message)`	Lance erreur avec message	`expect(() => fn()).toThrow('Erreur')`

Attention :
`.toThrow()` s'utilise uniquement avec des fonctions **synchrones**.
Pour les fonctions **asynchrones** (async / Promise), il faut utiliser
`await expect(promise).rejects.toThrow()`.

```
// Synchrone
expect(() => {
  throw new Error('Boom');
}).toThrow('Boom');

// Asynchrone
await expect(service.create({})).rejects.toThrow('Email existe
déjà');
```

5.6. Mocks – Vérifier les appels et isoler les dépendances

Un mock est une fausse implémentation d'une fonction ou d'un service, utilisée uniquement pour les tests afin d'éviter d'exécuter le vrai code.

Objectifs :

- ➔ Isoler le code testé de ses dépendances externes
- ➔ Rendre les tests rapides et prévisibles
- ➔ Simuler tous les cas (succès, erreurs, cas limites)

5.6.1. Créer un mock avec jest.fn()

jest.fn() crée une fonction fictive utilisée uniquement pour les tests, qui enregistre ses appels et peut retourner des valeurs personnalisées.

➢ **Exemple : Mock simple (sans retour)**

```
const mockFn = jest.fn();
mockFn('arg1', 'arg2');
expect(mockFn).toHaveBeenCalled();
expect(mockFn).toHaveBeenCalledWith('arg1', 'arg2');
```

➢ **Exemple : Mock avec valeur de retour**

```
const mockFn = jest.fn().mockReturnValue(42);
const result = mockFn();
expect(result).toBe(42);
expect(mockFn).toHaveBeenCalled();
```

➢ **Exemple : Mock avec Promise (succès)**

```
const mockFn = jest.fn().mockResolvedValue({ id: 1, nom: 'Dupont' });
```

```
const result = await mockFn();
expect(result.nom).toBe('Dupont');
expect(mockFn).toHaveBeenCalled();
```

➢ **Exemple : Mock avec Promise (erreur)**

```
const mockFn = jest.fn().mockRejectedValue(new Error('Erreur de
connexion'));
await expect(mockFn()).rejects.toThrow('Erreur de connexion');
expect(mockFn).toHaveBeenCalled();
```

➢ **Exemple : Mock avec implémentation personnalisée**

```
const mockFn = jest.fn().mockImplementation((a, b) => a + b);
const result = mockFn(2, 3);
expect(result).toBe(5);
expect(mockFn).toHaveBeenCalledWith(2, 3);
```

Méthode	Usage	Exemple
jest.fn()	Crée un mock vide	const mock = jest.fn()
.mockReturnValue(value)	Retour synchrone	mock.mockReturnValue(42)
.mockResolvedValue(value)	Promise réussie	mock.mockResolvedValue({ id: 1 })
.mockRejectedValue(error)	Promise échouée	mock.mockRejectedValue(new Error())
.mockImplementation(fn)	Comportement personnalisé	mock.mockImplementation((x) => x * 2)
.toHaveBeenCalled()	Fonction appelée	expect(mock).toHaveBeenCalled()
.toHaveBeenCalledTimes(n)	Appelée n fois	expect(mock).toHaveBeenCalledTimes(2)
.toHaveBeenCalledWith(args)	Appelée avec arguments	expect(mock).toHaveBeenCalledWith('test')

.not.toHaveBeenCalled()	Jamais appelée	expect(mock).not.toHaveBeenCalled()

5.6.2. Mocker un module entier avec jest.mock()

jest.mock() permet de remplacer tout un module (fichier importé) par des mocks. Utilisé pour les dépendances externes comme les bases de données, les API ou les services.

Différence avec jest.fn() :

- jest.fn() → Crée une fonction mock individuelle
- jest.mock() → Remplace tout un module (toutes ses fonctions/classes)

Exemple :

```
// jest.fn() : une fonction
const mockFn = jest.fn().mockReturnValue(42);
// jest.mock() : tout un fichier
jest.mock('../models/Visiteur'); // Remplace TOUTES les exports de ce fichier
```

1) Syntaxe obligatoire

jest.mock() doit être placé **AVANT les imports**

Exemple :

```
jest.mock('../models/Visiteur');
import { VisiteurModel } from '../models/Visiteur';
```

Quand vous faites jest.mock('../models/Visiteur'), **tout le contenu** du fichier est remplacé :

```
// Fichier original :   src/models/Visiteur.ts
export class VisiteurModel {
  static find() { /* code Mongoose */ }
  static findOne() { /* code Mongoose */ }
  constructor(data) { /* code Mongoose */ }
  save() { /* code Mongoose */ }
}

// Après jest.mock() :  tout est remplacé par des mocks vides
export class VisiteurModel {
  static find = jest.fn()      // Mock automatique
  static findOne = jest.fn()   // Mock automatique
  constructor = jest.fn()      // Mock automatique
  save = jest.fn()             // Mock automatique
}
```

Attention ! En pratique, Jest ne connaît pas les méthodes ajoutées dynamiquement par Mongoose.
Il peut être nécessaire de définir explicitement les méthodes mockées.

2) Configurer le comportement des mocks

Une fois le module mocké, il faut configurer ce qu'il retourne :

a) Méthode statique (appelée sur la classe)

```
// Configurer par exemple findOne() pour retourner une valeur
(VisiteurModel.findOne as jest.Mock).mockResolvedValue({ nom: 'Dupont' });
// Maintenant dans le code testé :
const result = await VisiteurModel.findOne({ email: 'test@test.com' });
// result = { nom: 'Dupont' }
```

b) Constructeur (appelé avec new)

Le constructeur crée une instance. Il faut mocker ce qu'il retourne :

```
// Créer un faux objet qui sera retourné par le constructeur
const mockInstance = {
  nom: 'Dupont',
  save: jest.fn().mockResolvedValue({ nom: 'Dupont' })
};
// Configurer le constructeur pour retourner ce faux objet
(VisiteurModel as any).mockImplementation(() => mockInstance);
// Maintenant dans le code testé :
const visiteur = new VisiteurModel({ nom: 'Dupont' });
// visiteur === mockInstance
await visiteur.save();
```

3) Les deux types de méthodes Mongoose

Mongoose a deux types de méthode* qu'il faut mocker différemment :

Type	Syntaxe	Comment mocker
Statique	`Model.findOne()`	(Model.findOne as jest.Mock).mockResolvedValue(...)
Instance	`instance.save()`	Mocker le constructeur avec mockImplementation

6. Mocker une dépendance externe (base de données)

Dans un service métier, certaines dépendances (base de données, API…) ne doivent **jamais** être appelées lors d'un test unitaire.

→ Si un **test** dépend d'un élément externe (base de données, API, fichier, réseau), alors on doit le **mocker**.

Le code suivant accède à une base de données MongoDB. Sans mock, il sera réellement exécuté pendant les tests.

```
const visiteur = await VisiteurModel.findOne({ email:
visiteurData.email });
```

Ce tableau montre la différence entre un test unitaire exécuté avec et sans mock :

❌ Sans Mock	✅ Avec Mock
Se connecte à MongoDB → Tests lents	Pas de base de données → Rapide
Crée des données en DB → Pollue et nécessite nettoyage	Pas de données créés
Dépend d'une connexion réseau → Fragile	Tests isolés → Testé UNIQUEMENT votre logique métier
Difficile de tester les erreurs → Comment forcer une erreur DB	Contrôle total → Simulation facile de tous les cas

Un test unitaire doit tester la logique métier, pas la base de données ni le réseau.

6.1. Analyse de la méthode createVisiteur

Code Métier à analyser

```
public async createVisiteur(visiteurData: ICreateVisiteur):
Promise<IVisiteurDocument> {
    try {
      // Vérifier si l'email existe déjà
      const existingVisiteur = await VisiteurModel.findOne({ email:
visiteurData.email });

      if (existingVisiteur) {
        throw new Error(`Un visiteur avec l'email
```

```
${visiteurData.email} existe déjà`);
      }
      // Créer et sauvegarder le visiteur
      const visiteur = new VisiteurModel(visiteurData);
      await visiteur.save();
      return visiteur;
    } catch (error: any) {
      // Gestion des erreurs de validation Mongoose
      if (error.name === 'ValidationError') {
        const messages = Object.values(error.errors).map((err: any)
=> err.message);
        throw new Error(`Validation échouée: ${messages.join(',
')}`);
      }
      throw error;
    }
  }
```

a) Signature de la méthode

```
public async createVisiteur(visiteurData: ICreateVisiteur):
Promise<IVisiteurDocument>
```

- Méthode asynchrone (async) → Retourne une Promise
- Type de retour : Promise<IVisiteurDocument> → Un objet Visiteur
- Paramètre : visiteurData de type ICreateVisiteur (données d'entrée)

b) Dépendances externes

La méthode dépend de Mongoose (base de données) : Il faut mocker Mongoose pour éviter d'appeler la vraie base de données.

Dépendance	Type	Ligne de code
`VisiteurModel.findOne()`	Méthode statique	`await VisiteurModel.findOne({ email: ... })`
`new VisiteurModel()`	Constructeur	`new VisiteurModel(visiteurData)`
`visiteur.save()`	Méthode d'instance	`await visiteur.save()`

c) Cas à tester

Cas nominal (succès)

Scénario : L'email est libre, la création réussie

Conditions :

- findOne() retourne null (email n'existe pas)
- save() réussit

Résultat attendu :

➔ Retourne un objet IVisiteurDocument
➔ findOne() a été appelé avec le bon email
➔ save() a été appelé

Cas d'erreur 1 : Email existe déjà

Scénario : Un visiteur avec cet email existe déjà

Conditions :

- findOne() retourne un objet (visiteur existant)

Résultat attendu :

➔ Lance une erreur avec le message : "Un visiteur avec l'email {email} existe déjà"
➔ save() ne doit **PAS** être appelé
➔ findOne() a été appelé

Cas d'erreur 2 : Erreur de validation Mongoose

Scénario : Les données sont invalides (ex: email mal formaté, champ requis manquant)

Conditions :

- findOne() retourne null
- save() rejette avec une erreur de type ValidationError

Résultat attendu :

➔ Lance une erreur avec le message : "Validation échouée: {détails}"
➔ Le message contient les erreurs formatées

6.2. Code des tests

Cas nominal (succès)

Scénario : L'email est libre, la création réussie

```
jest.mock('../models/Visiteur');

import { VisiteurService } from '../services/Visiteur';
import { VisiteurModel } from '../models/Visiteur';
import { ICreateVisiteur } from '../models/interfaces/IVisiteur';

describe('VisiteurService.createVisiteur', () => {
```

```
  let service: VisiteurService;

  beforeEach(() => {
    service = new VisiteurService();
    jest.clearAllMocks();
  });

  describe('Création d\'un visiteur réussie', () => {
    test('crée un visiteur si email libre et données valides',
async () => {
      // ARRANGE
      const visiteurData:  ICreateVisiteur = {
        nom: 'Dupont',
        prenom: 'Marie',
        email: 'marie.dupont@test.com',
        tel: '0612345678'
      };

      const expectedVisiteur = {
        ...visiteurData,
        _id: '507f1f77bcf86cd799439011',
        createdAt: new Date(),
        updatedAt:  new Date()
      };

      // Mock findOne :  email n'existe pas
      (VisiteurModel.findOne as
jest.Mock).mockResolvedValue(null);

      // Mock constructeur et save
      const mockVisiteurInstance = {
        ... expectedVisiteur,
        save: jest.fn().mockResolvedValue(expectedVisiteur)
      };

      (VisiteurModel as any).mockImplementation(() =>
mockVisiteurInstance);

      // ACT
      const result = await service.createVisiteur(visiteurData);
```

```
      // ASSERT
      expect(result).toBeDefined();
      expect(result.email).toBe('marie.dupont@test.com');
      expect(VisiteurModel.findOne).toHaveBeenCalledWith({ email:
'marie.dupont@test.com' });
      expect(mockVisiteurInstance.save).toHaveBeenCalled();
    });
  });
});
```